CentraleSupélec | université PARIS-SACLAY

PROJECT INFONUM 59 - Midpoint Review

AI for Assisting in the Analysis of Financial Results

GitHub

Céline Hudelot
Charles Moslonka
Hicham Randrianarivo

Gabriel Trier
Lucas Tramonte
Rayane Bouaita

December 2024

Contents

1 Introduction

1.1 Context

This project is part of a collaboration between the Artefact Research Center and the MICS Laboratory at CentraleSupélec. The focus lies on generative AI, its applications, and the development of digital commons.

The project is aligned with the France 2030 initiative, supported by the French government, in collaboration with Mistral, Giskard, the INA, and the BnF. The overarching goal is to create digital commons for the French generative AI ecosystem. This endeavor aims to deliver applied, impactful, and shared research, particularly in the domain of large language models (LLMs), a topic experiencing significant growth and innovation.

1.2 Project Objectives and Deliverables

The aim of this project is the design and implementation of an end-to-end LLM-based agent. This includes the following components:

- **Data Preparation and Enrichment:** Preparing, qualifying, and enhancing the initial dataset with the goal of making it open-source. This process will adhere to domain best practices and relevant regulations.
- **Evaluation of Existing Solutions:** Studying and selecting existing tools such as language models, prompting strategies, retrieval-augmented generation (RAG), and LLM agents to serve as foundational components for the agent's design.
- **Methodological Research:** Conducting research on reliability and robustness, with a focus on qualifying the validity and legitimacy of information provided by the agent, particularly regarding ESG (Environmental, Social, and Governance) compliance.
- **Agent Development:** Designing and developing the agent based on the selected methodologies and tools.
- **Evaluation and Deliverables:** Delivering a complete project, including the implementation of a GitHub repository with packaged algorithms shared under an open-source license.

The following diagram provides an overview of the project's key functionalities and expected outcomes.

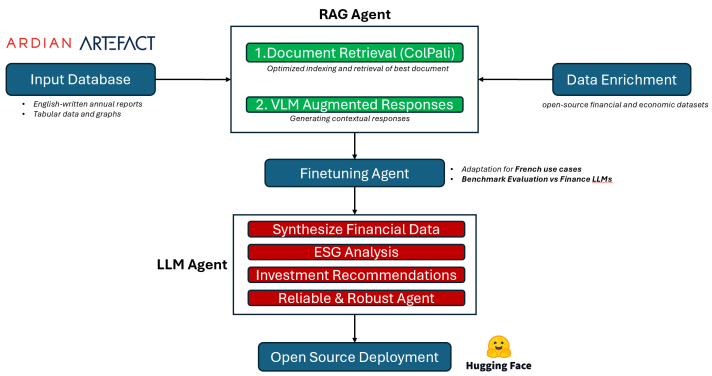


Figure 1: Project Diagram

This project aims to build a reliable and interpretable agent, advancing the state of generative AI while ensuring compliance with ESG standards and promoting collaboration in the AI community.

2 Project Organization

2.1 Tasks carried out

During the first phase of the project, we focused on **research and exploration of existing solutions**, as well as the **technical implementation of a ColPali-based RAG pipeline**. The following figure summarizes the tasks completed during this sprint:

Task name	Status	Assignee	Due	Priority
Lecture papier recherche Domain Specialization	Done	Gabriel Trier, Rayane Bouaita, Lucas Tramonte	7 novembre 2024	High
Prise en main Outils RAG langchain	Done	Gabriel Trier, Rayane Bouaita, Lucas Tramonte	7 novembre 2024	High
Appréhender jeu de données - ARDIAN x Artefact	Done	Lucas Tramonte, Rayane Bouaita, Gabriel Trier	11 novembre 2024	High
Recherche data set open source	Done	Lucas Tramonte	13 novembre 2024	Medium
Etat de l'art LLMs Finance	Done	Rayane Bouaita	13 novembre 2024	Medium
Adaptation RAG Langchain	Done	Gabriel Trier	13 novembre 2024	High
Statistique Dataset	Done	Rayane Bouaita, Lucas Tramonte	14 novembre 2024	High
User story - Cas d'usage	Done	Gabriel Trier	21 novembre 2024	Low
Documentation RAG Vision	Done	Rayane Bouaita, Lucas Tramonte, Gabriel Trier	22 novembre 2024	Medium
Statistique Zipf law	Done	Rayane Bouaita	3 décembre 2024	Medium
Ressources GPU pipeline	Blocage	Gabriel Trier, Lucas Tramonte	3 décembre 2024	High
Présentation vision projet: Artefact x MICS	Archived	Gabriel Trier, Lucas Tramonte, Rayane Bouaita	5 décembre 2024	High
Adapter l'approche RAG avec Colpali	Done	Gabriel Trier, Rayane Bouaita, Lucas Tramonte	11 décembre 2024	High
1 Setup DCE CentralSupélec	Done	Lucas Tramonte	11 décembre 2024	High
2 Tester Colpali/Colqwen avec VLM	In progress	Gabriel Trier, Rayane Bouaita	18 décembre 2024	Medium
3 GPU RAM modèle VLM	Blocage	Lucas Tramonte	18 décembre 2024	High
Exploration préprocessing dataset PDFS	In progress	Rayane Bouaita, Gabriel Trier	20 décembre 2024	Low
Rapport du projet	Done	Gabriel Trier, Lucas Tramonte, Rayane Bouaita	20 décembre 2024	Medium

Figure 2: Tasks Completed

2.2 Collaborative Tools

- Communication: Microsoft Teams, Notion
- Document and Report Sharing: Overleaf | Latex
- Code Repository: GitHub - Private, access available upon request.

3 Data Overview

The data used in this analysis comes from the company Ardian. The dataset consists of 107 annual reports, all of which are written in English and include a dedicated section on financial analysis. A statistical analysis was conducted on the data, revealing the following insights:

On average, each document is 78 pages long, which suggests that chunking the data by page could be an effective approach. Additionally, each document contains both tabular data and graphs, which should be made exploitable for further analysis. This highlights the need for a Retrieval-Augmented Generation (RAG) Vision model, and therefore, a Vision-Language Model (VLM) could be essential for this task.

The reports provide several types of exploitable information, including financial performance indicators, strategic plans, and growth initiatives. They also contain insights into sector trends and market conditions, as well as information on ESG (Environmental, Social, and Governance)

commitments. For instance, the reports highlight the carbon neutrality efforts of companies like *Emerson* and *Equity Trustees*. Finally, the documents cover risk management and governance practices, which are critical for understanding the companies' operations and strategic decisions.

3.1 User stories

We defined several user stories to guide the practical use cases of the agent and ensure it meets real-world needs. These user stories focus on key financial operations and decision-making processes.

3.1.1 Financial Summaries

- **Need**: The goal is to obtain concise financial summaries for internal follow-ups, corporate culture, and investor meetings in order to communicate company performance effectively.
- **LLM Agent**: The LLM will extract and synthesize key financial data, such as revenue, EBITA, margins, and provide a clear and concise summary.
- **Database**: The LLM will leverage the financial analysis section, which is presented in both tabular format and as general textual data.

3.1.2 ESG Practices Analysis

- **Need**: To obtain a detailed evaluation of environmental, social, and governance (ESG) practices of a company in order to align with its values.
- **LLM Agent**: The LLM will analyze ESG reports of companies in order to extract ESG initiatives (e.g., carbon emission reduction goals, diversity policies) and provide an assessment of their compliance with ESG standards.
- **Database**: The LLM will utilize the section on sustainable initiatives, carbon reduction goals (e.g., `EMR_2022` document), energy transition challenges, and environmental strategies (e.g., `ASX_88E` document).

3.1.3 Investment & Portfolio Diversification

- **Need**: To receive recommendations for diversifying a client's investments (or one's own) while minimizing risks, integrating high-growth sectors.
- **LLM Agent**: The LLM will analyze geographical revenue distribution and product diversification (e.g., `AMEX_EMR`), as well as strategic acquisitions and strategies (e.g., `ASX_EQT`).

4 State of the Art

4.1 LLMs Finance

With the rise of language models in specialized fields, finance has emerged as a key sector where these technologies can transform practices. Several models have been developed to address the specific needs of this industry, combining advanced architectures and high-quality financial data. Among them, BloombergGPT and FinBERT stand out for their approaches and capabilities.

BloombergGPT is a proprietary model developed by Bloomberg, designed to meet the specific requirements of the financial sector. This model is based on a Transformer architecture similar

to GPT and has been fine-tuned on an extensive corpus of financial data. These data, sourced from exclusive materials such as market reports, regulatory documents, and financial news articles, enable the model to grasp the nuances of financial language and concepts. BloombergGPT is used to generate financial reports, analyze complex regulatory texts, and automate the interpretation of market trends. However, its access remains limited as it is proprietary, which restricts its usage outside of Bloomberg-developed solutions.

FinBERT, on the other hand, is an open-source model based on BERT, specifically adapted for financial tasks. Trained on accessible data such as corporate reports and economic articles, it excels at tasks like sentiment classification—essential for determining whether a financial text is positive, negative, or neutral—and the extraction of entities specific to the domain. FinBERT is particularly valued for its effectiveness in targeted analyses, though it is less suited for text generation compared to models like BloombergGPT.

To evaluate the coherence and accuracy of the responses generated by the model we are developing, one of these specialized models, will be used as a reference to compare results and ensure a high level of quality and relevance.

4.2 Domain Specialization

Ling et al. (2023) argue that domain specialization plays a crucial role in enhancing the capabilities of Large Language Models (LLMs), enabling them to deliver disruptive outcomes across various industries. While general-purpose LLMs, such as GPT-based models, exhibit remarkable versatility, their performance in domain-specific tasks is often suboptimal due to the lack of fine-tuning on the specialized vocabulary, domain knowledge, and contextual understanding unique to particular fields. The authors advocate for the use of domain-adapted models, which are trained or fine-tuned on datasets that specifically reflect the terminology, structures, and nuances of the target domain. Such specialization allows the model to achieve greater accuracy, reliability, and interpretability when tasked with domain-specific queries [4].

In the context of financial analysis and ESG reporting, as outlined in the current project, domain specialization is particularly pertinent. Ling et al. emphasize the **inadequacies of general LLMs** in dealing with highly structured, domain-specific data, such as financial indicators (e.g., EBITDA, revenue growth, risk ratios) and ESG metrics (e.g., carbon emission reductions, diversity metrics, governance structures). These models are often ill-equipped to accurately interpret the contextual significance of such indicators without prior exposure to domain-specific knowledge. The authors suggest that specialized training data, incorporating both **quantitative data**(e.g., financial tables) and **qualitative data**(e.g., narrative reports), allows LLMs to perform better in understanding the intricate relationships between the different elements in financial documents and sustainability reports.

This research directly informed the approach taken in the project. To enhance the LLM's ability to analyze financial and ESG reports, the system was designed to incorporate domain-specific fine-tuning. The focus was on adapting the model to better interpret sector-specific metrics, sustainability goals, and governance structures typically found in financial documents. Following the methodology described by Ling et al., the LLM agent was fine-tuned on a carefully curated dataset of financial reports and ESG disclosures, improving its ability to identify and synthesize the most relevant insights, especially within the context of specific companies or industries.

Moreover, Ling et al. emphasize the importance of incorporating domain-specific data not only

during model training but also in the **retrieval and augmentation stages**. The combination of a specialized retrieval-augmented generation (RAG) architecture with a Vision-Language Model (VLM) is in line with their findings that such hybrid models are better suited for tasks requiring a deep understanding of both textual and visual data. In this project, the integration of ColPali for document retrieval, coupled with Qwen-VL for response generation, **aligns with Ling et al.'s recommendation to combine advanced retrieval techniques with domain-adapted LLMs** to achieve superior results in specialized tasks.

By leveraging domain-specialized models and incorporating relevant datasets, this approach aims to significantly improve the model's accuracy in extracting and interpreting critical financial and ESG data. This directly aligns with Ling et al.'s assertion that domain adaptation is not only necessary for improving the interpretability and performance of LLMs but is also pivotal for ensuring the reliability and robustness of the insights generated by such models in high-stakes domains like finance and sustainability.

4.3 Data Enrichment

In order to enrich the existing dataset, several open-source financial and economic datasets were explored. Initially, two datasets were chosen as benchmarks due to their relevance and accessibility:

1. Finance Alpaca provides a collection of financial market data, including quotes, historical prices, and various financial indicators for assets. The dataset has been cleaned and is available for use in its processed form at Wealth Alpaca Dataset Additionally, a GitHub repository with performance analysis, training scripts, data generation scripts, and inference notebooks can be found here.

2. Financial Reports SEC Dataset provides free access to the financial reports of publicly listed companies in the United States, as filed with the Securities and Exchange Commission (SEC). It includes balance sheets, income statements, and other regulatory filings.

In addition to these benchmark datasets, the OSCAR Dataset was considered for the translation of documents into languages other than English. OSCAR is a multilingual dataset sourced from web content and is widely used for natural language processing (NLP), machine translation, and AI research. This dataset can be accessed here

Furthermore, several other datasets, while less central to the project, could serve as additional sources of enrichment for the dataset. These include:

- ECB SDW Dataset (European Central Bank Statistical Data Warehouse) offers extensive statistical data on the economic and monetary policies of the European Central Bank (ECB). It includes detailed data on inflation, interest rates, monetary aggregates, and other economic indicators from the Eurozone and its member states.

- OECD Economic Outlook Dataset provides economic forecasts and analyses of global trends for OECD member countries. It includes macroeconomic indicators, growth projections, unemployment rates, and reports on international economic policies.

- INSEE (Institut National de la Statistique et des Études Économiques) publishes official statistical data for France, covering a wide range of sectors such as the economy, population, employment, and more. The dataset includes national and regional economic indicators, as well as census data.

- CNet Dataset is a dataset of web text data gathered from Common Crawl, a project that collects data from the web for NLP and machine learning applications. This dataset is available for

download, although it often requires preprocessing to be useful in specific applications.

By combining these datasets, the dataset's scope and diversity can be significantly expanded, facilitating more robust analysis and offering a broader set of insights into financial performance, market trends, and other related topics.

5 Technical Pipeline

5.1 Getting Started with LangChain

5.1.1 Retrieval-Augmented Generation (RAG)

One of our initial objectives was to familiarize ourselves with commonly used tools to build a custom Retrieval Augmented Generation (RAG) pipeline and test it with the project's dataset.

A standard RAG pipeline is composed of two primary parts:

1. **Indexing:** This step involves preprocessing the dataset by splitting documents into manageable chunks, generating vector embeddings for these chunks, and storing them in a vector database for efficient retrieval.
2. **Retrieval:** At query time, relevant chunks are retrieved from the vector database based on semantic similarity to the input query. These chunks are then passed to a language model for response generation.

Figure 3 below illustrates a typical RAG pipeline, highlighting the main components and their interactions.

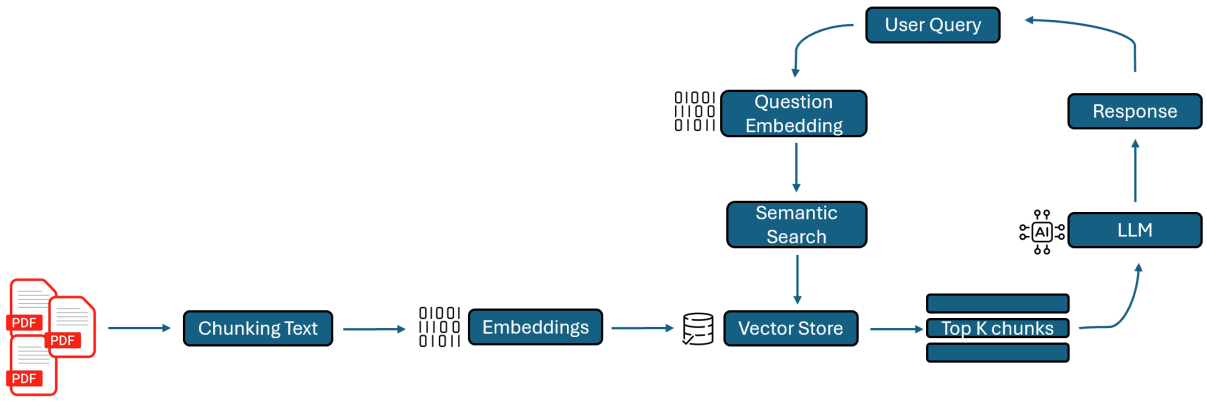


Figure 3: Schema of a classic RAG Pipeline.

5.1.2 Our LangChain Integration

LangChain is a modular framework designed to simplify the development of AI-driven pipelines by integrating a variety of tools.

In our experimental pipeline, we leveraged LangChain's flexibility to process the document corpus of our dataset and generate responses to specific queries. We used the following key elements provided by LangChain:

- **Document Loaders:** Tools to load and extract content from different document formats. In our case, we extracted data from PDF files.
- **Text Splitters:** Documents were split into smaller chunks to enable efficient processing.
- **Embeddings:** Convert text chunks into numerical vectors to enable semantic search.
- **Vector Stores:** Embeddings were stored in a FAISS (Facebook AI Similarity Search) index for rapid and efficient retrieval.
- **Retrievers:** Fetch relevant document chunks based on the input query.
- **LLMs (Language Models):** We used the Flan-T5 model for query interpretation and response generation, as it effectively handles French-language queries. Due to hardware limitations, we couldn't use larger models like Mistral for this preliminary exploration.
- **Chains:** Orchestrate the workflow, managing the flow of information between components.

The figure below (Figure 4) illustrates an example output of our pipeline when tested on a specific query.

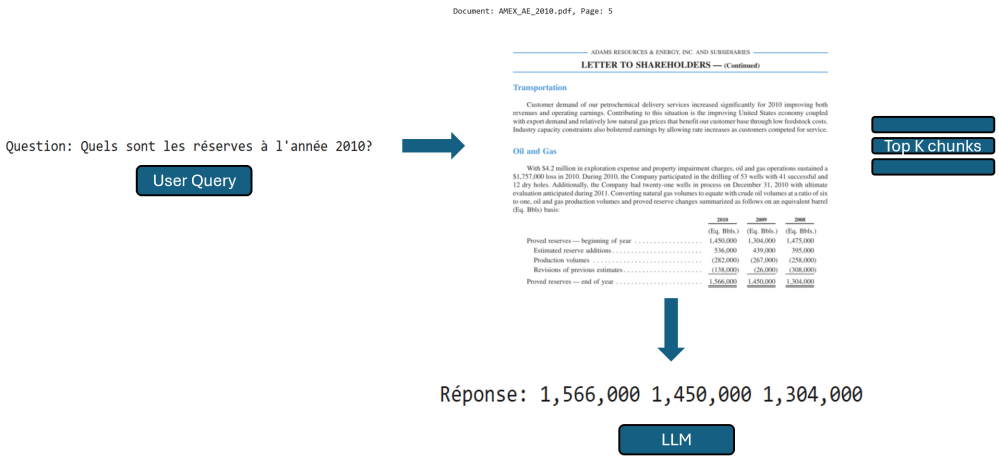


Figure 4: Traditional RAG Vision Pipeline

We can observe that the pipeline successfully retrieves the correct document and locates the page containing the relevant information. However, it struggles processing poorly structured tabular and textual data, which are key limitations. These limitations restrict its applicability in scenarios requiring a multimodal understanding of the data.

5.2 RAG Vision approach

To overcome these challenges, we explored Retrieval-Augmented Generation (RAG) Vision as an alternative. This approach combines information retrieval and generative models to deliver precise and contextually enriched responses. RAG Vision systems retrieve relevant documents or visual data and use them to answer questions, generate summaries, or provide context-aware insights. This makes them highly effective for domains like finance, healthcare, legal analysis, and education, where handling both structured and unstructured data is essential.

5.2.1 Challenges in RAG vision

Despite their potential, nowadays RAG vision systems face significant limitations when dealing with visually rich documents. These challenges include:

- **Text-Centric Approaches:** Traditional pipelines heavily rely on Optical Character Recognition (OCR) to extract text, often neglecting the rich visual context embedded in charts, layouts, and infographics.
- **Inefficient Preprocessing:** Separate steps like OCR, layout detection, and chunking introduce latency and increase the computational cost, limiting scalability for large datasets.
- **Limited Multi-Modal Integration:** Existing systems struggle to fully integrate visual and textual features, reducing their effectiveness for tasks requiring a holistic understanding of document content.

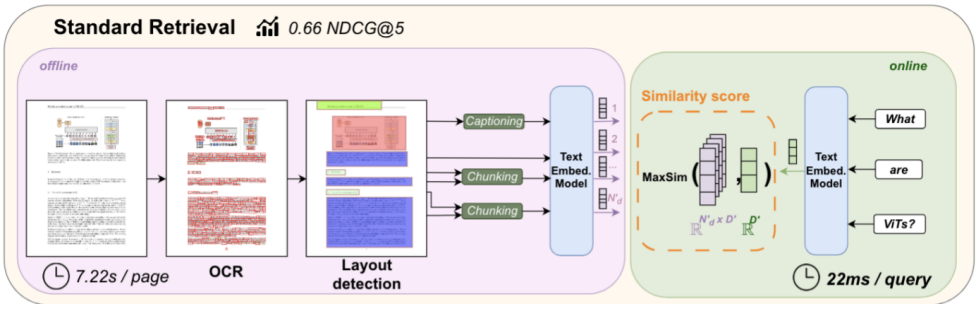


Figure 5: Tradition RAG Vision Pipeline

5.2.2 ColPali

ColPali is an innovative retrieval system designed to overcome the challenges of visually rich document retrieval. Unlike traditional pipelines that rely heavily on OCR and text-based indexing, ColPali leverages the latest advancements in Vision Language Models (VLMs) to directly embed document page images and perform efficient query matching using late interaction mechanisms.[1]

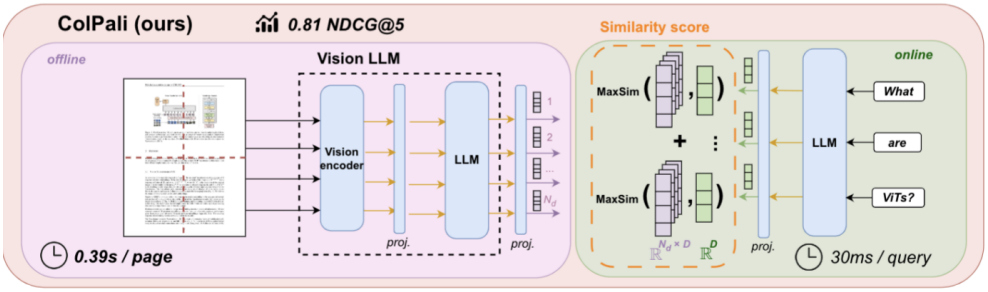


Figure 6: Colpali Pipeline

5.2.3 Indexing Phase

The indexing phase in ColPali aims to simplify traditional document ingestion pipelines by processing each page of the document as an image directly.

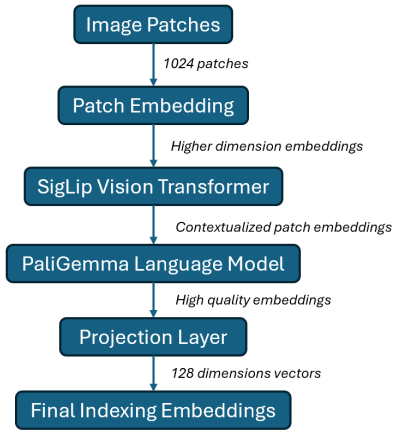


Figure 7: Schema of indexing Colpali pipeline

Below is a detailed breakdown of each step in the indexing phase:

1. Image Patch Extraction The first phase consists in dividing each image into a grid of smaller image patches. This process ensures that both local and global visual information, such as spatial structures, layouts, and visual elements like charts or tables, is captured.

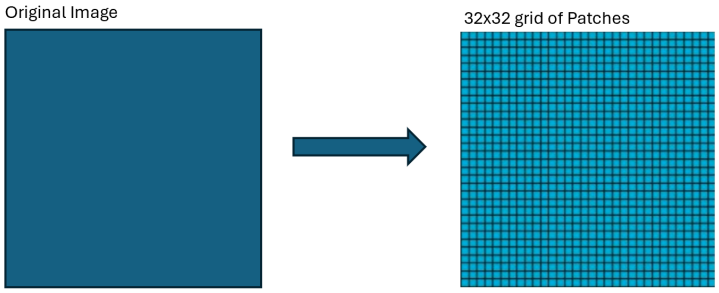


Figure 8: Schema of image patching process

2. Patch Embedding The individual image patches are then embedded into high-dimensional vectors, to represent their visual features.

- A linear projection layer is applied to each image patch to transform the raw pixel data into meaningful patch embeddings.
- This step ensures that each patch is represented in a way that highlights its unique visual characteristics.

3. Vision Transformer (SigLIP-So400m) The resulting patch embeddings are processed by a Vision Transformer (ViT), the SigLIP-So400m model, to capture complex relationships between the patches. [2]

- The Vision Transformer models the interaction between patches using self-attention mechanisms.
- It encodes spatial and structural relationships between different regions of the document.

The output of this process is a contextualized patch embeddings, which incorporate both local and global context within the document. This step ensures that the relationships between the patches are fully understood.

4. Language Model (Gemma-2B) The contextualized patch embeddings are inputted as "soft tokens" into the PaliGemma-2B language model, a Vision-Language Model (VLM), to align the visual features with semantic textual representations.

The language model processes the contextualized patch embeddings as if they were language tokens, producing high-quality embeddings that integrate both visual and semantic context.

5. Final Patch Embeddings Finally, the embeddings from the language model are projected into a lower-dimensional latent space ($D = 128$) for efficient storage and faster retrieval. This lower dimensional output is a multi-vector representation of the document page, where each vector corresponds to a specific patch.

5.2.4 Retrieval Phase

The retrieval phase in ColPali efficiently matches user queries to relevant document pages using a late interaction mechanism. This phase focuses on leveraging the pre-indexed embeddings for fast and precise query matching.

1. Query Embedding At runtime, the user query is embedded by the Gemma-2B Language Model into a series of token embeddings, each representing a specific term in the query.

2. Late Interaction Mechanism ColPali employs a ColBERT-style late interaction (LI) approach to match query tokens with document patches. For each term in the query, the system identifies the document patch that has the most similar representation based on the ColPali embeddings. The late interaction mechanism ensures that every term in the query interacts with all document patches. It combines the strengths of bi-encoder models (offline computation and fast retrieval) with the detailed matching capabilities of cross-encoders. [3] [2]

Late Interaction. Given a query q and a document d, let $\mathbf{E}_q \in \mathbb{R}^{N_q \times D}$ and $\mathbf{E}_d \in \mathbb{R}^{N_d \times D}$ represent their respective multi-vector embeddings in the shared embedding space $\mathbb{R}^D$. The late interaction operator $LI(q, d)$ is computed as:

$$LI(q, d) = \sum_{i=1}^{N_q} \max_{j=1}^{N_d} \langle \mathbf{E}_q^{(i)}, \mathbf{E}_d^{(j)} \rangle$$

With:
- $\mathbf{E}_q^{(i)}$ is the i-th query embedding vector.
- $\mathbf{E}_d^{(j)}$ is the j-th document embedding vector.
- $\langle \cdot, \cdot \rangle$ denotes the dot product.

Here is an illustration of the late interaction process : [3]

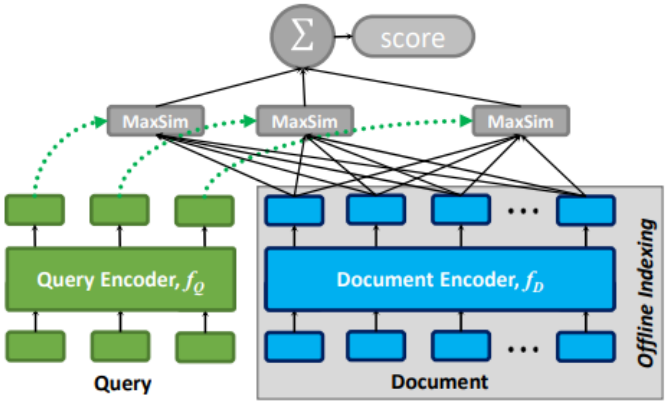


Figure 9: Illustration of Late Interaction Mechanism in ColPali

3. Query to Document Scoring For each query token, the document patch with the highest similarity score is selected. The final query-to-document score is computed by summing these maximum scores across all query tokens. This ensures a comprehensive interaction between the query and document, incorporating both semantic and visual context.

5.2.5 Performances

ColPali is benchmarked on the ViDoRe dataset, which evaluates retrieval systems on tasks involving visually rich documents across domains such as healthcare, finance, and academia. ViDoRe tasks include retrieving tables (TabFQuAD), scientific figures (ArxivQA), and infographics (InfoVQA), providing a comprehensive test of ColPali's capabilities.

Dataset	# Queries	Domain
Academic Tasks		
DocVQA (eng)	500 (500)	Industrial
InfoVQA (eng)	500 (500)	Infographics
TAT-DQA (eng)	1600 (1600)	Varied Modalities
arXiVQA (eng)	500 (500)	Scientific Figures
TabFQuAD (fra)	210 (210)	Tables
Practical Tasks		
Energy (eng)	100 (1000)	Scientific
Government (eng)	100 (1000)	Administrative
Healthcare (eng)	100 (1000)	Medical
AI (eng)	100 (1000)	Scientific
Shift Project (fra)	100 (1000)	Environment

Figure 10: Composition of the ViDoRe benchmark.

ColPali demonstrates state-of-the-art performance on the ViDoRe benchmark, significantly outperforming all baseline models. [2]

- **High Retrieval Performance:** ColPali outperforms existing models on diverse tasks such as tables (TabFQuAD), infographics (InfoVQA), and scientific figures (ArxivQA), with an average NDCG@5 score of 81.3, compared to 67.0 for the closest baseline.

	ArxivQ	DocQ	InfoQ	TabF	TATQ	Shift	AI	Energy	Gov.	Health.	Avg.
Unstructured Text only											
- BM25	-	34.1	-	-	44.0	59.6	90.4	78.3	78.8	82.6	-
- BGE-M3	-	28.4 $_{\downarrow 5.7}$	-	-	36.1 $_{\downarrow 7.9}$	68.5 $_{\uparrow 8.9}$	88.4 $_{\downarrow 2.0}$	76.8 $_{\downarrow 1.5}$	77.7 $_{\downarrow 1.1}$	84.6 $_{\uparrow 2.0}$	-
Unstructured + OCR											
- BM25	31.6	36.8	62.9	46.5	62.7	64.3	92.8	85.9	83.9	87.2	65.5
- BGE-M3	31.4 $_{\downarrow 0.2}$	25.7 $_{\downarrow 11.1}$	60.1 $_{\downarrow 2.8}$	70.8 $_{\uparrow 24.3}$	50.5 $_{\downarrow 12.2}$	**73.2** $_{\uparrow 8.9}$	90.2 $_{\downarrow 2.6}$	83.6 $_{\downarrow 2.3}$	84.9 $_{\uparrow 1.0}$	91.1 $_{\uparrow 3.9}$	66.1 $_{\uparrow 0.6}$
Unstructured + Captioning											
- BM25	40.1	38.4	70.0	35.4	61.5	60.9	88.0	84.7	82.7	89.2	65.1
- BGE-M3	35.7 $_{\downarrow 4.4}$	32.9 $_{\downarrow 5.4}$	71.9 $_{\uparrow 1.9}$	69.1 $_{\uparrow 33.7}$	43.8 $_{\downarrow 17.7}$	73.1 $_{\uparrow 12.2}$	88.8 $_{\uparrow 0.8}$	83.3 $_{\downarrow 1.4}$	80.4 $_{\downarrow 2.3}$	91.3 $_{\uparrow 2.1}$	67.0 $_{\uparrow 1.9}$
Contrastive VLMs											
Jina-CLIP	25.4	11.9	35.5	20.2	3.3	3.8	15.2	19.7	21.4	20.8	17.7
Nomic-vision	17.1	10.7	30.1	16.3	2.7	1.1	12.9	10.9	11.4	15.7	12.9
SigLIP (Vanilla)	43.2	30.3	64.1	58.1	26.2	18.7	62.5	65.7	66.1	79.1	51.4
Ours											
SigLIP (Vanilla)	43.2	30.3	64.1	58.1	26.2	18.7	62.5	65.7	66.1	79.1	51.4
BiSigLIP (+fine-tuning)	58.5 $_{\uparrow 15.3}$	32.9 $_{\uparrow 2.6}$	70.5 $_{\uparrow 6.4}$	62.7 $_{\uparrow 4.6}$	30.5 $_{\uparrow 4.3}$	26.5 $_{\uparrow 7.8}$	74.3 $_{\uparrow 11.8}$	73.7 $_{\uparrow 8.0}$	74.2 $_{\uparrow 8.1}$	82.3 $_{\uparrow 3.2}$	58.6 $_{\uparrow 7.2}$
BiPali (+LLM)	56.5 $_{\downarrow 2.0}$	30.0 $_{\downarrow 2.9}$	67.4 $_{\downarrow 3.1}$	76.9 $_{\uparrow 14.2}$	33.4 $_{\uparrow 2.9}$	43.7 $_{\uparrow 17.2}$	71.2 $_{\downarrow 3.1}$	61.9 $_{\downarrow 11.7}$	73.8 $_{\downarrow 0.4}$	73.6 $_{\downarrow 5.8}$	58.8 $_{\uparrow 0.2}$
ColPali (+Late Inter.)	**79.1** $_{\uparrow 22.6}$	**54.4** $_{\uparrow 24.5}$	**81.8** $_{\uparrow 14.4}$	**83.9** $_{\uparrow 7.0}$	**65.8** $_{\uparrow 32.4}$	**73.2** $_{\uparrow 29.5}$	**96.2** $_{\uparrow 25.0}$	**91.0** $_{\uparrow 25.1}$	**92.7** $_{\uparrow 18.9}$	**94.4** $_{\uparrow 20.8}$	**81.3** $_{\uparrow 22.5}$

Figure 11: Results on Vidore Benchmark

- **Faster Indexing:** By bypassing the need for complex preprocessing pipelines like OCR and layout detection, ColPali reduces indexing latency by a factor of 18, compared to standard retrieval systems.

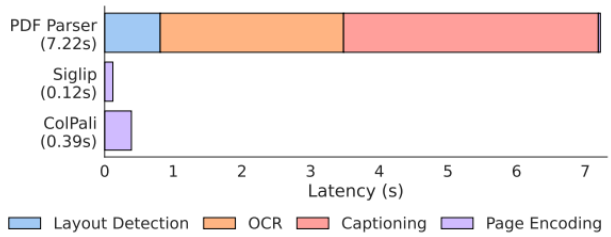
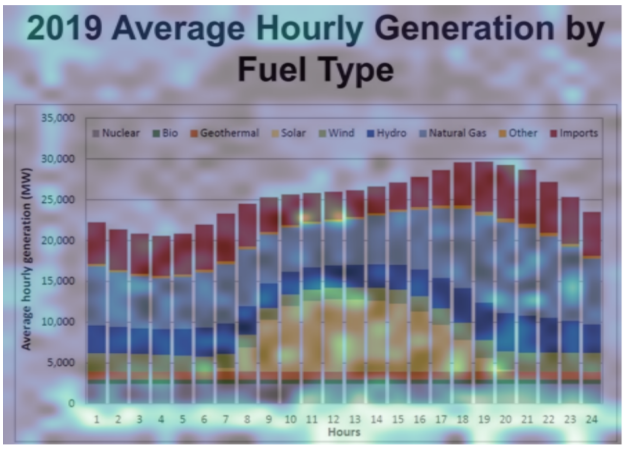


Figure 12: Offline indexing performances

5.2.6 Conclusion

To conclude, ColPali stands out as a powerful tool for visually rich document retrieval, particularly in our project context, where interpretability and explainability are key concerns.



Query: "Which hour of the day had the highest overall eletricity generation in 2019?"

Figure 13: Interpretability of the most relevant document image patches for query.

Its ability to generate visual heatmaps, as shown in Figure 13, allows us to identify which document patches are most relevant to a given query. [1]

Beyond interpretability, ColPali is well-suited for our goals:

- **Performance in Specific Domains:** It excels in tasks involving finance and other highly structured fields, aligning with the project's requirements.
- **Modular Integration:** We can leverage ColPali for the indexing phase, efficiently creating contextualized embeddings from visually rich documents. For the retrieval phase, a complementary LLM or VLM can provide additional context and enhance the quality of responses.

However, certain challenges must be addressed:

- **Limited Vector Store Support:** Current vector databases lack native support for multi-vector embeddings, requiring on-disk storage solutions.
- **Training Limitations:** ColPali is trained predominantly in English, which might lead to performance drops in French domain-specific applications.

6 Final Pipeline Description

Our final pipeline consists of two main stages, document retrieval and VLM augmented answers.

In the first part, we leverage the Colpali model to retrieve the most relevant documents corresponding to the user query.

In the second part, we use Qwen-VL, a powerful Vision-Language Model (VLM) to generate accurate and contextualized answers by combining the retrieved documents and the user query within a structured prompt.

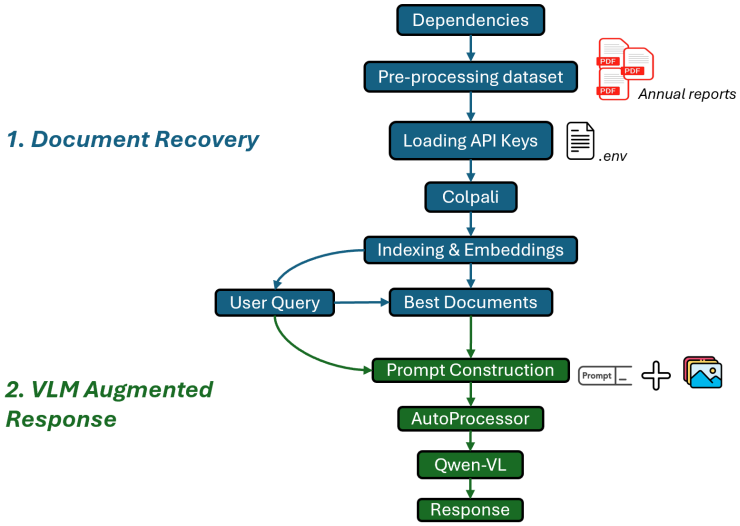


Figure 14: Overview of the Final Pipeline Architecture

6.1 Pipeline Steps

The pipeline is divided into the following steps:

- **Dependencies and Initialization:** We load the necessary libraries, ColPALI model for document retrieval, Qwen-VL for response generation, and API keys.
- **Document Indexing:** Using ColPALI, the input PDFs are pre-processed and indexed. This step prepares the documents for fast retrieval based on query similarity.
- **Query-Based Retrieval:** Given a user query, the most relevant documents are identified using cosine similarity between query embeddings and document embeddings.
- **Interpretability with Similarity Maps:** For the top-k retrieved documents, we generate token-based similarity maps to highlight the regions most relevant to the query. These maps provide a visual representation of how the model interprets the input.
- **Response Generation:** The user query and the most relevant documents are combined into a structured prompt. This prompt is passed to Qwen-VL, which generates a contextualized and accurate response.

7 Challenges Encountered

During the initial phase of the project, we faced **several challenges** that impacted our progress:

1. **Data Availability and Future Requirements**

   While the initial dataset from *Ardian* (see Section 4.3) provided a foundation, we identified the need to source additional, well-labeled, and meaningful datasets to ensure their relevance for future model testing and deployment.

2. **Computational Resource Constraints**

Our initial LangChain pipeline (see Section 5.1.2) was implemented on *Google Colab*, where limited GPU resources forced us to use smaller, less capable LLMs. This affected both computational performance and graphical processing capabilities. Transitioning to Centrale's *DCE* introduced further challenges, particularly when integrating ColPali with a Vision-Language Model (VLM), which further delayed progress.

3. **Uncertainty in Use Cases**

   Confirmation of the use cases with *Ardian*, planned for January, may require adjustments to technical choices based on potential new requirements.

4. **RAG Methodology Learning Curve**

   Mastering Retrieval-Augmented Generation (RAG) techniques required time and effort, slightly delaying the initial development phases.

These challenges highlight areas for improvement as the project advances.

8 Next Steps

The next steps of the project will focus on the following functionalities and tasks:

- **Prioritize RAG Vision Pipeline Implementation:**
  - Implement and finalize the RAG Vision pipeline, focusing on the retrieval of relevant documents from PDFs and the integration of these documents into the response generation process.
  - Ensure that the ColPali model for document retrieval and the Qwen-VL model for visual answer generation are fully functional and optimized for accurate responses based on the PDF content.
  - Implement an efficient method for indexing PDFs in a way that reduces GPU memory consumption on DCE. This will be crucial for ensuring scalability and performance during the document retrieval and response generation process.
- **Refine LLM Agent's Core Capabilities:**
  - Enhance the LLM agent's ability to synthesize financial data, ESG practices, and provide contextual recommendations, particularly focusing on financial summaries and ESG analysis.
  - Improve the LLM's ability to generate investment and portfolio diversification recommendations based on company strategies.
- **Optimize RAG Methodology:**
  - Fine-tune the Retrieval-Augmented Generation (RAG) model to better retrieve, rank, and integrate the most relevant documents from the dataset.
  - Address any issues with the integration of ColPali with the Vision-Language Model (VLM), ensuring seamless document retrieval and accurate generation of visual responses.
- **Data Enrichment (Lower Priority):**
  - While data enrichment will remain an ongoing task, it is currently of lower priority compared to the pipeline implementation.

  - Focus on sourcing and integrating additional open-source datasets that could enhance the LLM agent's capabilities for future use cases, but only after the core functionality is in place.
- **Deployment and Testing:**
  - Once the core functionality of the agent is stable, focus on preparing the system for deployment and conducting comprehensive end-to-end testing.
  - Ensure that the agent is fully optimized for real-world use, scalable, and capable of handling the demands of the use cases.
- **GitHub Repository and Documentation:**
  - Organize the GitHub repository with all algorithms, code, and detailed documentation for public access.
  - Make the project open source with Hugging Face.

References

[1] Manuel Faysse. Analysis of the colpali: Efficient document retrieval with vision language models paper. `https://huggingface.co/blog/manu/colpali`, 2024. Hugging Face Blog, accessed on 2024-12-18.

[2] Manuel Faysse, Hugues Sibille, Tony Wu, Bilel Omrani, Gautier Viaud, Céline Hudelot, and Pierre Colombo. Colpali: Efficient document retrieval with vision language models. *arXiv preprint arXiv:2407.01449*, 2024. Available at `https://arxiv.org/abs/2407.01449`.

[3] Omar Khattab and Matei Zaharia. Colbert: Efficient and effective passage search via contextualized late interaction over bert. *arXiv preprint arXiv:2004.12832*, 2020. Available at `https://arxiv.org/abs/2004.12832`.

[4] Chen Ling, Xujiang Zhao, Jiaying Lu, Chengyuan Deng, Can Zheng, Junxiang Wang, Tanmoy Chowdhury, Yun Li, Hejie Cui, Xuchao Zhang, et al. Domain specialization as the key to make large language models disruptive: A comprehensive survey. *arXiv preprint arXiv:2305.18703*, 2023.